

```

import math

import numpy as np
import plotly.express as px
import scipy
from database import MongoRepository

from statsmodels.stats.contingency_tables import Table2x2
from statsmodels.stats.power import GofChisquarePower
from teaching_tools.ab_test.experiment import Experiment

# Tasks 7.4.7, 7.4.9, 7.4.10, 7.4.19
class GraphBuilder:
    """Methods for building Graphs."""

    def __init__(self, repo=MongoRepository()):
        """init

        Parameters
        -----
        repo : MongoRepository, optional
            Data source, by default MongoRepository()
        """
        self.repo = repo

    def build_nat_choropleth(self):
        """Creates nationality choropleth map.

        Returns
        -----
        Figure
        """
        # Get nationality counts from database
        df_nationality = self.repo.get_nationality_value_counts(normalize=True)

        # Create Figure
        fig = px.choropleth(
            data_frame=df_nationality,
            locations="country_iso3",
            color="count_pct",
            projection="natural earth",
            color_continuous_scale= px.colors.sequential.Oranges,
            title = "DS Applicants: Nationality"
        )
        return fig

        # Return Figure
        pass

    def build_age_hist(self):
        """Create age histogram.

        Returns
        -----
        Figure
        """
        # Get ages from repository
        ages= self.repo.get_ages()

        # Create Figure
        fig = px.histogram(x=ages, nbins=20, title = "DS Applicants: Distribution of Ages")
        fig.update_layout(xaxis_title="Age", yaxis_title = "Frequency [count]")
        # Return Figure

```

```
    return fig
```

```
def build_ed_bar():
    """Creates education level bar chart.

    Returns
    -----
    Figure
    """
    # Get education level value counts from repo
    education = self.repo.get_ed_value_counts(normalize=True)
    # Create Figure
    fig = px.bar(
        x=education,
        y=education.index,
        orientation="h",
        title="DS Applicants: Highest Degree Earned"
    )
    # Add axis labels
    fig.update_layout(xaxis_title="Frequency [count]", yaxis_title="Degree")
    return fig
    # Return Figure
    pass
```

```
def build_contingency_bar(self):
    """Creates side-by-side bar chart from contingency table.

    Returns
    -----
    Figure
    """
    # Get contingency table data from repo
    data = self.repo.get_contingency_table()

    # Create Figure
    fig = px.bar(
        data_frame=data,
        barmode="group",
        title= "Admissions Quiz Completion by Group"
    )
    # Set axis labels
    fig.update_layout(
        xaxis_title= "Group",
        yaxis_title= "Frequency [count]",
        legend= {"title": "Admissions Quiz" }
    )
    return fig
```

```
# Tasks 7.4.12, 7.4.18, 7.4.20
```

```
class StatsBuilder:
    """Methods for statistical analysis."""

    def __init__(self, repo=MongoRepository()):
        """init

        Parameters
        -----
        repo : MongoRepository, optional
            Data source, by default MongoRepository()
        """
        self.repo=repo

    def calculate_n_obs(self, effect_size):
        """Calculate the number of observations needed to detect effect size.
```

Parameters

effect_size : float

Effect size you want to be able to detect

Returns

int

Total number of observations needed, across two experimental groups.

"""

Calculate group size, w/ alpha=0.05 and power=0.8

chi_square_power = GofChisquarePower()

group_size = math.ceil(chi_square_power.solve_power(effect_size=effect_size, alpha=0.05,
power=0.8)

)

Return number of observations (group size * 2)

return group_size * 2

def calculate_cdf_pct(self, n_obs, days):

"""Calculate percent chance of gathering specified number of observations in
specified number of days.

Parameters

n_obs : int

Number of observations you want to gather.

days : int

Number of days you will run experiment.

Returns

float

Percentage chance of gathering ``n\_obs`` or more in ``days``.

"""

Get data from repo

no_quiz = self.repo.get_no_quiz_per_day()

Calculate quiz per day mean and std

mean = no_quiz.describe()["mean"]

std = no_quiz.describe()["std"]

Calculate mean and std for days

sum_mean = mean * days

sum_std = std * np.sqrt(days)

Calculate CDF probability, subtract from 1

prob = 1 - scipy.stats.norm.cdf(n_obs, loc = sum_mean, scale = sum_std)

Turn probability to percentage

pct = prob * 100

Return percentage

return pct

def run_experiment(self, days):

"""Run experiment. Add results to repository.

Parameters

days : int

Number of days to run experiment for.

"""

Instantiate Experiment

exp = Experiment(repo=self.repo, db="wqu-abtest", collection="ds-applicants")

```
# Reset experiment
exp.reset_experiment()

# Run Experiment
result = exp.run_experiment(days=days)
```

```
def run_chi_square(self):
    """Tests nominal association.

    Returns
    -----
    A bunch containing the following attributes:

    statistic: float
        The chi^2 test statistic.

    df: int
        The degrees of freedom of the reference distribution

    pvalue: float
        The p-value for the test.
    """
    # Get data from repo
    data = self.repo.get_contingency_table()

    # Create `Table2X2` from data
    contingency_table = Table2x2(data.values)

    # Run chi-square test
    chi_square_test = contingency_table.test_nominal_association()
    # Return chi-square results
    return chi_square_test
```